

1.

| Topic                                    | Decision                                                                                                                                                                   |
|------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Object-oriented programming language     | <b>Python:</b> I chose Python because it is easy to understand and supports object-oriented programming. I will use classes and objects to organize the Solitaire game.    |
| GUI library                              | <b>Tkinter:</b> I chose Tkinter because it is included with Python and can be used to create buttons, labels, check boxes, radio buttons, lines, and other GUI components. |
| IDE (Integrated Development Environment) | <b>Visual Studio Code (VS Code).</b> I chose VS Code because I am familiar with it and it supports Python development, debugging, and testing.                             |
| xUnit framework                          | <b>Python unittest.</b> I chose unittest because it is included with Python and provides the features needed to create and run unit tests.                                 |
| Programming style guide                  | <b>Google Python Style Guide.</b> I will follow the guide for naming, indentation, comments, functions, classes, and general Python coding practices.                      |
| Project hosting site                     | Github.com                                                                                                                                                                 |
| Other decisions                          | I will develop the Solitaire game using OOP and separate the game logic from the GUI as the project becomes more complete.                                                 |

2. class SolitaireBoard:

```
def __init__(self, marbles=32):  
    self.marbles = marbles
```

```
def remove_marble(self):  
    if self.marbles > 0:  
        self.marbles -= 1
```

```
def get_marble_count(self):  
    return self.marbles
```

```

import unittest

from solitaire import SolitaireBoard

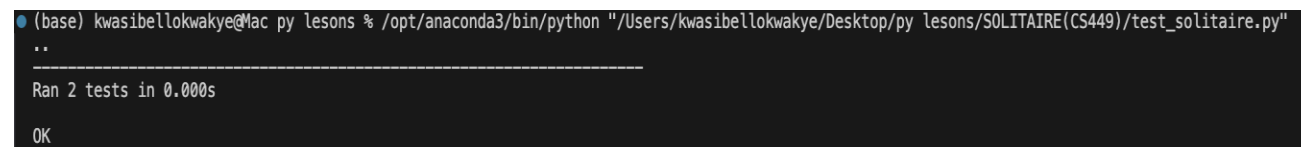
class TestSolitaireBoard(unittest.TestCase):

    def test_initial_marble_count(self):
        board = SolitaireBoard()
        self.assertEqual(board.get_marble_count(), 32)

    def test_remove_marble(self):
        board = SolitaireBoard()
        board.remove_marble()
        self.assertEqual(board.get_marble_count(), 31)

if __name__ == "__main__":
    unittest.main()

```



A terminal window showing the execution of a Python script. The prompt is '(base) kwasibellokwakye@Mac py lessons %'. The command executed is '/opt/anaconda3/bin/python "/Users/kwasibellokwakye/Desktop/py lessons/SOLITAIRE(CS449)/test\_solitaire.py"'. The output shows 'Ran 2 tests in 0.000s' and 'OK'.

```

(base) kwasibellokwakye@Mac py lessons % /opt/anaconda3/bin/python "/Users/kwasibellokwakye/Desktop/py lessons/SOLITAIRE(CS449)/test_solitaire.py"
..
Ran 2 tests in 0.000s
OK

```

3.

```

import tkinter as tk

def start_game():
    status_label.config(text="New game started!")

window = tk.Tk()
window.title("Solitaire")
window.geometry("500x400")

```

```
# Text
title_label = tk.Label(
    window,
    text="Peg Solitaire",
    font=("Arial", 20)
)
title_label.pack(pady=10)

instruction_label = tk.Label(
    window,
    text="Choose a board type and start a new game."
)
instruction_label.pack()

# Radio buttons
board_type = tk.StringVar(value="English")

english_button = tk.Radiobutton(
    window,
    text="English",
    variable=board_type,
    value="English"
)
english_button.pack()

diamond_button = tk.Radiobutton(
    window,
    text="Diamond",
    variable=board_type,
```

```
        value="Diamond"
    )
    diamond_button.pack()

    hexagon_button = tk.Radiobutton(
        window,
        text="Hexagon",
        variable=board_type,
        value="Hexagon"
    )
    hexagon_button.pack()

# Check box
record_game = tk.BooleanVar()

record_checkbox = tk.Checkbutton(
    window,
    text="Record game",
    variable=record_game
)
record_checkbox.pack(pady=10)

# Line
line = tk.Frame(window, height=2, bg="black")
line.pack(fill="x", padx=40, pady=10)

# Button
new_game_button = tk.Button(
    window,
```

```
        text="New Game",
        command=start_game
    )
new_game_button.pack(pady=10)

# Status text
status_label = tk.Label(
    window,
    text="Ready to play."
)
status_label.pack(pady=10)

window.mainloop()
```

